

Chat with PDFs: a Multimodal RAG System

Architecture, Technology Stack & System Design

Project: github.com/Divvy18s/chat_with_pdfs (2023 → 2026 rebuild)

September 2026

Contents

1	What was built	2
2	Technology stack	2
3	System architecture	3
3.1	Ingestion pipeline (upload → indexed)	3
3.2	Query pipeline (question → streamed answer)	4
3.3	Deployment (Docker Compose)	4
4	Step-by-step: technology, function, handling	4
4.1	Step 1 – Upload and chat sessions	4
4.2	Step 2 – Extraction (text, tables, images)	4
4.3	Step 3 – Vision enrichment (pixels to words)	5
4.4	Step 4 – Content-aware chunking	5
4.5	Step 5 – Dense and vision indexing	5
4.6	Step 6 – Hybrid retrieval with RRF	5
4.7	Step 7 – Prompt and generation	5
4.8	Step 8 – Streaming and persistence	5
4.9	Step 9 – Deletion cascade	6
5	System design decisions (interview-ready)	6
6	Research foundations	6
7	Verification performed	6
8	Thirty-second pitch	7
A	API reference	7

1 What was built

The 2023 prototype was a single-file Streamlit demo (`app.py`): PyPDF2 text dump → fixed 1000/200 character chunks → in-memory FAISS → `ConversationalRetrievalChain`. It lost all data on refresh, had no users, no citations, and blocked the UI while indexing.

The 2026 rebuild is a full-stack, multi-user, multimodal RAG product:

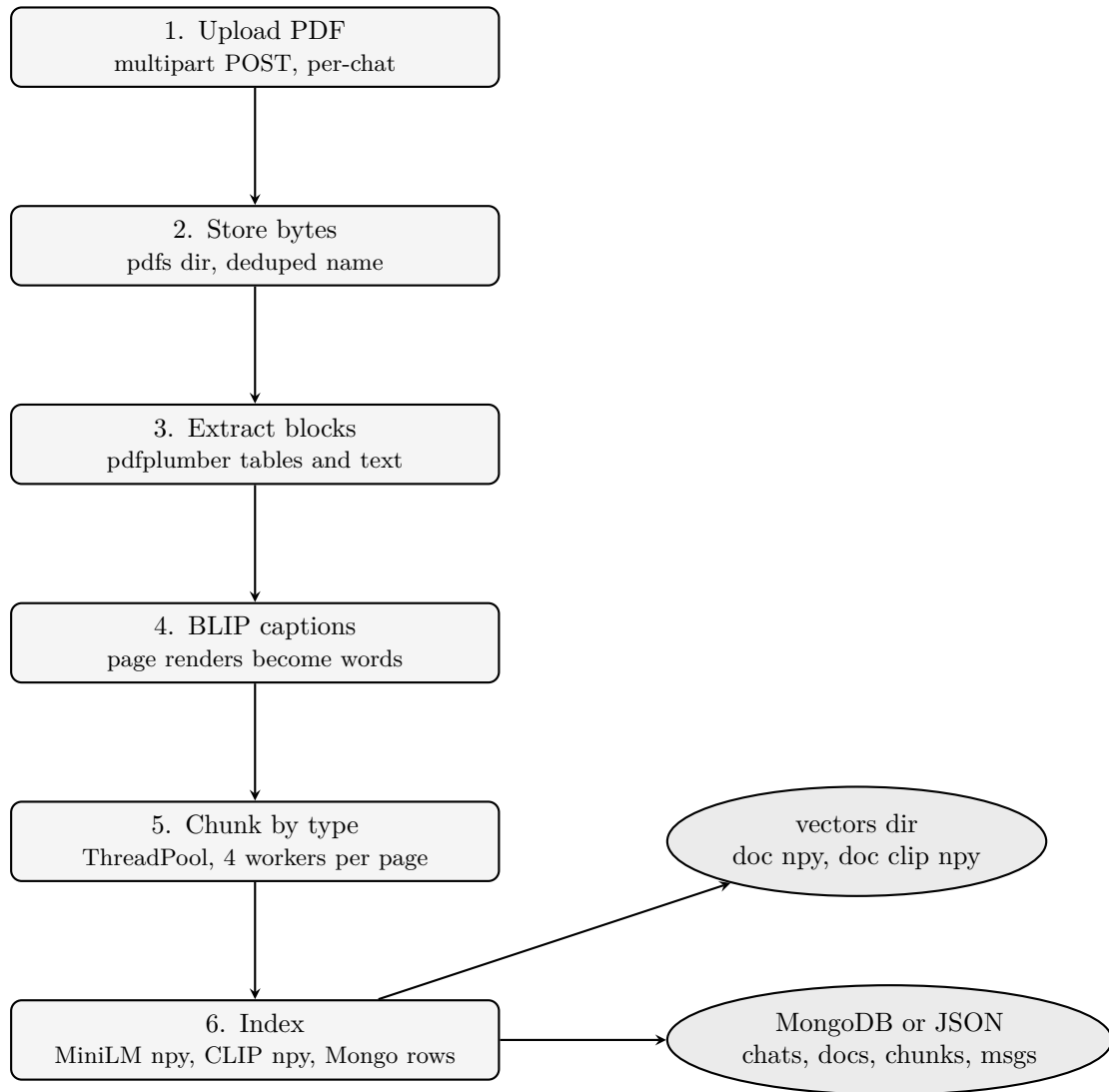
Concern	2023	2026
Frontend	Streamlit tutorial page	ChatGPT-style 3-pane UI + Next.js 15 scaffold
Backend	none (in-process)	Django 6 + Django Ninja (typed, SSE)
Database	none	MongoDB (local-JSON fallback)
Retrieval	in-memory FAISS	Hybrid RRF: MiniLM + TF-IDF + CLIP
Chunking	fixed 1000/200 chars	content-aware: text / table / code / image
Vision	none	CLIP vectors + BLIP captions
LLM	ChatOpenAI default	Groq gpt-oss-120b (env-switchable)
Ingest	blocking button	ThreadPool pages × files; Celery in Docker
Sessions	none	per-chat workspaces, history, delete cascade
Deploy	<code>streamlit run</code>	Docker Compose (api, worker, mongo, redis)

2 Technology stack

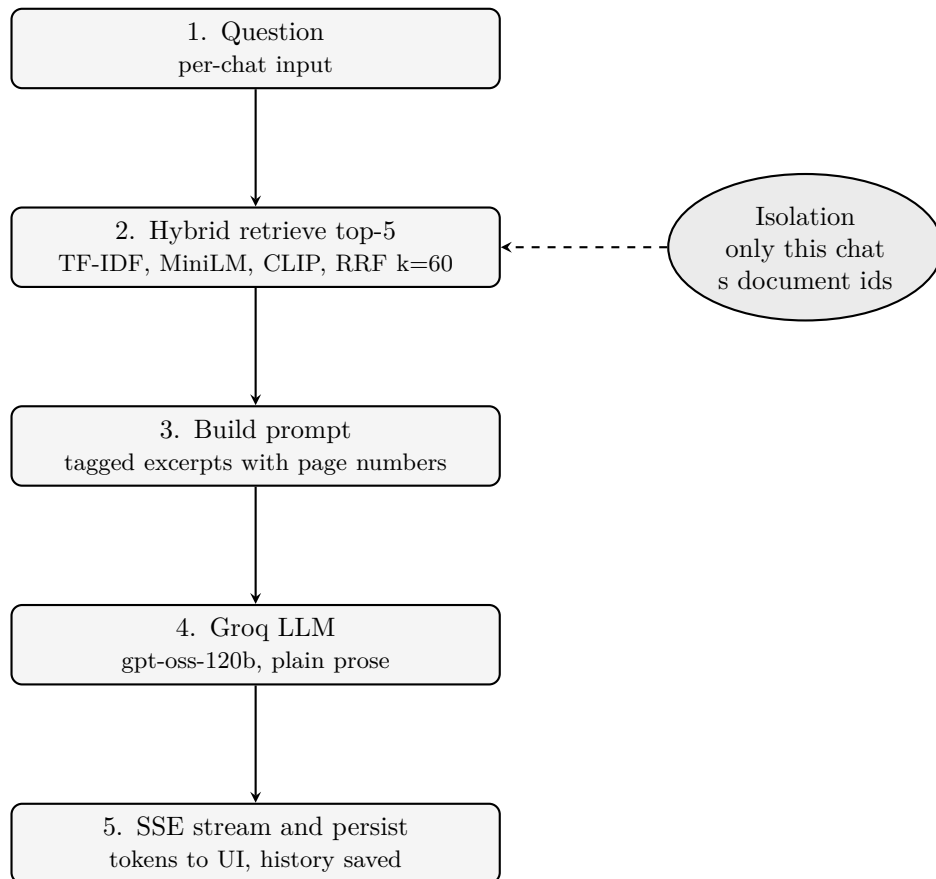
Layer	Technology	Role
UI	Django templates + Tailwind; Next.js 15 + TS (scaffold)	3-pane chat: history, conversation, per-chat upload
API	Django 6, django-ninja 1.7	typed REST + Server-Sent Events streaming
DB	MongoDB 7 (pymongo); local JSON fallback	chats, documents, chunks, messages (audit trail)
PDF text	pdfplumber 0.11, pypdf 6, PyMuPDF 1.28	text + table extraction, image census + rendering
Sparse	scikit-learn TF-IDF (1-2 grams)	keyword signal, always available
Dense	sentence-transformers 6, MiniLM-L6-v2 384-d; torch 2.14 CPU	semantic signal, <code>.npy</code> per document
Vision	CLIP ViT-B/32; BLIP-base (transformers 5)	image vectors + pixels-to-words captions
LLM	Groq OpenAI REST (urllib , no SDK)	answers, gpt-oss-120b default
Concurrency	ThreadPoolExecutor ; Celery + Redis 7 (Docker)	parallel chunking + multi-file ingest
Deploy	Docker Compose; Node 20 image	reproducible multi-service stack
Docs	LaTeX (Tectonic) + TikZ	this architecture document

3 System architecture

3.1 Ingestion pipeline (upload → indexed)



3.2 Query pipeline (question → streamed answer)



3.3 Deployment (Docker Compose)

API and worker share one image with different commands (stateless, horizontally scalable). State lives in volumes: `mongo_data` (documents/chunks/history), vector files, PDF dir. `frontend:3000` → `api:8000` → `mongo:27017`, `redis:6379`; `docker compose up --scale worker=3` demonstrates ingestion scaling.

4 Step-by-step: technology, function, handling

4.1 Step 1 – Upload and chat sessions

Tech: Django Ninja File upload; `chats` collection (`db.py`).

Does: creates one workspace per chat holding its own `doc_ids`; filenames deduped on disk.

Handles: non-PDF rejected (400); unknown chat id → 404; multi-file endpoint fans out on threads and reports per-file errors plus `total_ms`.

4.2 Step 2 – Extraction (text, tables, images)

Tech: `pdfplumber` primary, threaded `pypdf` fallback, `PyMuPDF` census (`extract_blocks`).

Does: emits typed blocks `{kind: text|table|image, page, text}`; tables serialized to GitHub-flavour markdown.

Handles: borderless tables fall back to text (still page-cited); missing `pdfplumber` → `pypdf` path; corrupt PDF → clean 400, nothing half-indexed.

4.3 Step 3 – Vision enrichment (pixels to words)

Tech: PyMuPDF 72dpi page renders; BLIP-base captioning; page-text excerpts.

Code: `extract_page_images`, `caption_images`. **Does:** each image block gains `Visual: <caption>` plus up to 400 chars of neighbouring page text.

Handles: no-image pages skipped; BLIP failure → caption omitted, CLIP + proxy still work (`USE_CAPTION=0` disables).

4.4 Step 4 – Content-aware chunking

Tech: recursive splitter + heading prefix; whole-table rule; code line rule (`chunk_block`).

Does: text 800/120 chars keeping sentence/paragraph boundaries; tables exactly one chunk (never split mid-row); code 600/80 preserving lines; images one chunk.

Handles: code detected by indentation/keywords; borderless tabular text caught by column heuristic; tiny fragments (<20 chars) dropped.

4.5 Step 5 – Dense and vision indexing

Tech: MiniLM-L6-v2 (384-d, normalized) and CLIP ViT-B/32 (512-d), NumPy row stores (`embeddings.py`).

Does: writes `vectors/{doc}.numpy` aligned to chunk idx and `{doc}.clip.numpy` aligned to image chunks; upload response reports `dense/dim/vision`.

Handles: encoder/model failure → sparse-only mode; old docs without `.numpy` encoded on the fly once; `USE_DENSE=0/USE_VISION=0` force fallbacks.

4.6 Step 6 – Hybrid retrieval with RRF

Tech: TF-IDF cosine + MiniLM cosine + CLIP cosine fused by Reciprocal Rank Fusion, `k=60` (`hybrid_order`); optional CrossEncoder rerank.

Does: three rank lists vote; visual-intent queries (`colour/figure/chart/show words`) with confident CLIP (>0.20) pin image hits first; text queries unaffected.

Handles: verified – “red square” returns the image chunk first, “capital of France” the text chunk; any signal can fail closed while the rest answer.

4.7 Step 7 – Prompt and generation

Tech: Groq OpenAI-compatible REST via `urllib` (no SDK pin risk); model from env (`groq_client.py`).

Does: top-5 excerpts tagged `[TABLE]/[TEXT]/[CODE]/[IMAGE]` with page numbers; system prompt demands plain ChatGPT-style prose, no citation markers.

Handles: missing key → explicit error; Groq-side decommissioned models surfaced with 400 text (Qwen was retired mid-project; default moved to `gpt-oss-120b`).

4.8 Step 8 – Streaming and persistence

Tech: `StreamingHttpResponse` SSE stream; history saved *before* streaming (`chat_stream_s`).

Does: word tokens stream into the bubble; on `[DONE]` the UI reloads persisted history – nothing vanishes; SSE errors show a hint instead of silent loss.

Handles: server restart mid-stream → `onerror` message pointing at `/api/health`.

4.9 Step 9 – Deletion cascade

Tech: DELETE /api/chats/id → db.delete_chat (+ embedding cleanup).

Does: removes chat, documents, chunks, messages, PDF bytes, MiniLM and CLIP vectors. Verified: other chats untouched.

Handles: asks confirmation in UI; Mongo and JSON-fallback paths both covered.

5 System design decisions (interview-ready)

1. **Stateless API + stateful stores.** Django/Celery containers hold no state; MongoDB, vector files, PDFs live in volumes. Scale workers without data loss.
2. **Tenant isolation per chat.** Every retrieval filters by `chat.doc_ids` (proven: finance chat cannot see Paris notes). Same pattern as SaaS tenant scoping.
3. **Async-friendly ingest.** ThreadPool today (`pages × files`, `total_ms` reported), Celery+Redis in Docker for true background jobs with retries.
4. **Graceful degradation ladder:** vision → dense → sparse. A missing GPU/model never pages the user.
5. **Audit trail.** All Q/A pairs persisted per chat – replayable, evaluable, demoable.
6. **Three-signal retrieval over one index.** Avoids operating Qdrant locally while keeping the RRF story; Qdrant HNSW is the named production swap.
7. **Secrets hygiene.** Bare-gsk_ .env accepted locally; .env.example documents Docker format; keys never baked into images.

6 Research foundations

Paper / method	What we took from it
RAG, Lewis et al. 2020	retrieve-then-generate baseline architecture
Self-RAG, Asai et al. 2023	self-grading + retry loop (documented stretch)
Corrective RAG, Yan et al. 2024	rewrite/retry on weak evidence (stretch)
HyDE, Gao et al. 2022	hypothetical-answer retrieval for vague queries (stretch)
Lost in the Middle, Liu et al. 2023	justification for reranking top hits
Late Chunking, J. Günther et al. (Jina AI) 2024	embed-then-chunk analysed; deferred (needs long-context encoder)
RAPTOR 2024	hierarchical summaries (deferred)
RRF, Cormack et al. 2009	k=60 fusion of our three rank lists
CLIP, Radford et al. 2021	shared text-image space for visual retrieval
BLIP, Li et al. 2022	bootstrapped captioning for pixels-to-words
ColPali/ColQwen, Faysse et al. 2024	page-level vision embeddings (deferred, scan-only use case)

7 Verification performed

manage.py check clean; upload/index/chat/delete exercised via Django test client: health {dense:true, dim:384, vision:true}; grid-table PDF → {text:1, table:1}; “Q1 revenue?” answered with both text+table hits; “red square” → image chunk first; “capital of France” → text first; chat delete removed PDF + both .npy files leaving other chats intact; SSE persistence bug reproduced and fixed (save-before-stream).

8 Thirty-second pitch

“ChatGPT for PDFs, rebuilt as a real system: per-chat workspaces with isolated documents, a threaded multimodal ingest that treats text, tables, code and figures differently, and a three-signal hybrid retriever – MiniLM dense plus TF-IDF sparse plus CLIP vision fused with RRF – with BLIP captions so the LLM can describe images. Stateless Django API, MongoDB audit trail, SSE streaming, Docker Compose for scale-out.”

A API reference

GET /api/health, GET/POST /api/chats, GET/DELETE /api/chats/{id}, POST /api/chats/{id}/upload, GET .../documents, GET .../messages, POST .../chat, GET .../stream?q=; legacy global /api/documents, /api/chat, /api/chat/stream, POST /api/documents/upload-many. Flags: GROQ_MODEL, EMBED_MODEL, USE_DENSE, USE_VISION, USE_CAPTION, USE_RERANK, MONGO_URI, RAG_TOP_K.